Double Buffer

Intent

Cause a series of sequential operations to appear instantaneous or simultaneous.

Motivation

In their hearts, computers are sequential beasts. Their power comes from being able to break down the largest tasks into tiny steps that can be performed one after another. Often, though, our users need to see things occur in a single instantaneous step or see multiple tasks performed simultaneously.

> With threading and multi-core architectures this is becoming less true, but even with several cores, only a few operations are running concurrently.

A typical example, and one that every game engine must address, is rendering. When the game draws the world the users see, it does so one piece at a time — the mountains in the distance, the rolling hills, the trees, each in its turn. If the user *watched* the view draw incrementally like that, the illusion of a coherent world would be shattered. The scene must update smoothly and quickly, displaying a series of complete frames, each appearing instantly.

Double buffering solves this problem, but to understand how, we first need to review how a computer displays graphics.

How computer graphics work (briefly)

A video display like a computer monitor draws one pixel at a time. It sweeps across each row of pixels from left to right and then moves down to the next row. When it reaches the bottom right corner, it scans back up to the top left and starts all over again. It does this so fast — around sixty times a second — that our eyes can't see the scanning. To us, it's a single static field of colored pixels — an image.

> This explanation is, err, "simplified". If you're a low-level hardware person and you're cringing right now, feel free to skip to the next section. You already know enough to understand the rest of the chapter. If you *aren't* that person, my goal here is to give you just enough context to understand the pattern we'll discuss later.

You can think of this process like a tiny hose that pipes pixels to the display. Individual colors go into the back of the hose, and it sprays them out across the display, one bit of color to each pixel in its turn. So how does the hose know what colors go where?

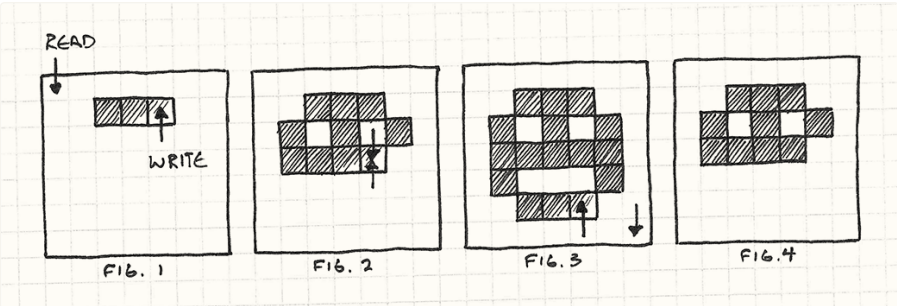
In most computers, the answer is that it pulls them from a *framebuffer*. A framebuffer is an array of pixels in memory, a chunk of RAM where each couple of bytes represents the color of a single pixel. As the hose sprays across the display, it reads in the color values from this array, one byte at a time.

> The specific mapping between byte values and colors is described by the *pixel format* and the *color depth* of the system. In most gaming consoles today, each pixel gets 32 bits: eight each for the red, green, and blue channels, and another eight left over for various other purposes.

Ultimately, in order to get our game to appear on screen, all we do is write to that array. All of the crazy advanced graphics algorithms we have boil down to just that: setting byte values in the framebuffer. But there's a little problem.

Earlier, I said computers are sequential. If the machine is executing a chunk of our rendering code, we don't expect it to be doing anything else at the same time. That's mostly accurate, but a couple of things *do* happen in the middle of our program running. One of those is that the video display will be reading from the framebuffer *constantly* while our game runs. This can cause a problem for us.

Let's say we want a happy face to appear on screen. Our program starts looping through the framebuffer, coloring pixels. What we don't realize is that the video driver is pulling from the framebuffer right as we're writing to it. As it scans across the pixels we've written, our face starts to appear, but then it outpaces us and moves into pixels we haven't written yet. The result is *tearing*, a hideous visual bug where you see half of something drawn on screen.



> We start drawing pixels just as the video driver starts reading from the framebuffer (Fig. 1). The video driver eventually catches up to the renderer and then races past it to pixels we haven't written yet (Fig. 2). We finish drawing (Fig. 3), but the driver doesn't catch those new pixels.
>
> The result (Fig. 4) is that the user sees half of the drawing. The name "tearing" comes from the fact that it looks like the bottom half was torn off.

This is why we need this pattern. Our program renders the pixels one at a time, but we need the display driver to see them all at once — in one frame the face isn't there, and in the next one it is. Double buffering solves this. I'll explain how by analogy.

Act 1, Scene 1

Imagine our users are watching a play produced by ourselves. As scene one ends and scene two starts, we need to change the stage setting. If we have the stagehands run on after the scene and start dragging props around, the illusion of a coherent place will be broken. We could dim the lights while we do that

(which, of course, is what real theaters do), but the audience still knows *something* is going on. We want there to be no gap in time between scenes.

With a bit of real estate, we come up with this clever solution: we build *two* stages set up so the audience can see both. Each has its own set of lights. We'll call them stage A and stage B. Scene one is shown on stage A. Meanwhile, stage B is dark as the stagehands are setting up scene two. As soon as scene one ends, we cut the lights on stage A and bring them up on stage B. The audience looks to the new stage and scene two begins immediately.

At the same time, our stagehands are over on the now darkened stage *A*, striking scene one and setting up scene *three*. As soon as scene two ends, we switch the lights back to stage A again. We continue this process for the entire play, using the darkened stage as a work area where we can set up the next scene. Every scene transition, we just toggle the lights between the two stages. Our audience gets a continuous performance with no delay between scenes. They never see a stagehand.

> Using a half-silvered mirror and some very smart layout, you could actually build this so that the two stages would appear to the audience in the same *place*. As soon as the lights switch, they would be looking at a different stage, but they would never have to change where they look. Building this is left as an exercise for the reader.

Back to the graphics

That is exactly how double buffering works, and this process underlies the rendering system of just about every game you've ever seen. Instead of a single framebuffer, we have *two*. One of them represents the current frame, stage A in our analogy. It's the one the video hardware is reading from. The GPU can scan through it as much as it wants whenever it wants.

> Not *all* games and consoles do this, though. Older and simpler consoles where memory is limited carefully sync their drawing to the video refresh instead. It's tricky.

Meanwhile, our rendering code is writing to the *other* framebuffer. This is our darkened stage B. When our rendering code is done drawing the scene, it switches the lights by *swapping* the buffers. This tells the video hardware to start reading from the second buffer now instead of the first one. As long as it times that switch at the end of a refresh, we won't get any tearing, and the entire scene will appear all at once.

Meanwhile, the old framebuffer is now available for use. We start rendering the next frame onto it. Voilà!

The Pattern

A **buffered class** encapsulates a **buffer**: a piece of state that can be modified. This buffer is edited incrementally, but we want all outside code to see the edit as a single atomic change. To do this, the class keeps *two* instances of the buffer: a **next buffer** and a **current buffer**.

When information is read *from* a buffer, it is always from the *current* buffer. When information is written *to* a buffer, it occurs on the *next* buffer. When the changes are complete, a **swap** operation swaps the next and current buffers instantly so

that the new buffer is now publicly visible. The old current buffer is now available to be reused as the new next buffer.

When to Use It

This pattern is one of those ones where you'll know when you need it. If you have a system that lacks double buffering, it will probably look visibly wrong (tearing, etc.) or will behave incorrectly. But saying, "you'll know when you need it" doesn't give you much to go on. More specifically, this pattern is appropriate when all of these are true:

- We have some state that is being modified incrementally.
- That same state may be accessed in the middle of modification.
- We want to prevent the code that's accessing the state from seeing the work in progress.
- We want to be able to read the state and we don't want to have to wait while it's being written.

Keep in Mind

Unlike larger architectural patterns, double buffering exists at a lower implementation level. Because of this, it has fewer consequences for the rest of the codebase—most of the game won't even be aware of the difference. There are a couple of caveats, though.

The swap itself takes time

Double-buffering requires a *swap* step once the state is done being modified. That operation must be atomic—no code can access *either* state while they are being swapped. Often, this is as quick as assigning a pointer, but if it takes longer to swap than it does to modify the state to begin with, then we haven't helped ourselves at all.

We have to have two buffers

The other consequence of this pattern is increased memory usage. As its name implies, the pattern requires you to keep *two* copies of your state in memory at all times. On memory-constrained devices, this can be a heavy price to pay. If you can't afford two buffers, you may have to look into other ways to ensure your state isn't being accessed during modification.

Sample Code

Now that we've got the theory, let's see how it works in practice. We'll write a very bare-bones graphics system that lets us draw pixels on a framebuffer. In most consoles and PCs, the video driver provides this low-level part of the graphics system, but implementing it by hand here will let us see what's going on. First up is the buffer itself:

```
class Framebuffer
{
public:
  Framebuffer() { clear(); }
```

```
  void clear()
  {
    for (int i = 0; i < WIDTH * HEIGHT; i++)
    {
      pixels_[i] = WHITE;
    }
  }

  void draw(int x, int y)
  {
    pixels_[(WIDTH * y) + x] = BLACK;
  }

  const char* getPixels()
  {
    return pixels_;
  }

private:
  static const int WIDTH = 160;
  static const int HEIGHT = 120;

  char pixels_[WIDTH * HEIGHT];
};
```

It has basic operations for clearing the entire buffer to a default color and setting the color of an individual pixel. It also has a function, `getPixels()`, to expose the raw array of memory holding the pixel data. We won't see this in the example, but the video driver will call that function frequently to stream memory from the buffer onto the screen.

We wrap this raw buffer in a `Scene` class. It's job here is to render something by making a bunch of `draw()` calls on its buffer:

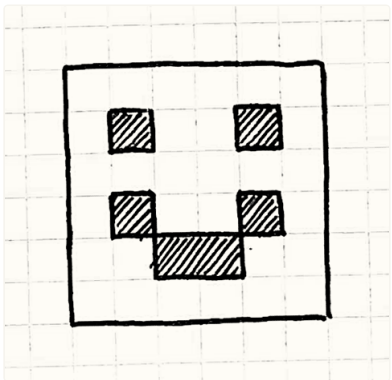
```
class Scene
{
public:
  void draw()
  {
    buffer_.clear();

    buffer_.draw(1, 1);
    buffer_.draw(4, 1);
    buffer_.draw(1, 3);
    buffer_.draw(2, 4);
    buffer_.draw(3, 4);
    buffer_.draw(4, 3);
  }

  Framebuffer& getBuffer() { return buffer_; }

private:
  Framebuffer buffer_;
};
```

Specifically, it draws this artistic masterpiece:



Every frame, the game tells the scene to draw. The scene clears the buffer and then draws a bunch of pixels, one at a time. It also provides access to the internal buffer through `getBuffer()` so that the video driver can get to it.

This seems pretty straightforward, but if we leave it like this, we'll run into problems. The trouble is that the video driver can call `getPixels()` on the buffer at *any* point in time, even here:

```
buffer_.draw(1, 1);
buffer_.draw(4, 1);
// <- Video driver reads pixels here!
buffer_.draw(1, 3);
buffer_.draw(2, 4);
buffer_.draw(3, 4);
buffer_.draw(4, 3);
```

When that happens, the user will see the eyes of the face, but the mouth will disappear for a single frame. In the next frame, it could get interrupted at some other point. The end result is horribly flickering graphics. We'll fix this with double buffering:

```
class Scene
{
public:
  Scene()
  : current_(&buffers_[0]),
    next_(&buffers_[1])
  {}

  void draw()
  {
    next_->clear();

    next_->draw(1, 1);
    // ...
    next_->draw(4, 3);

    swap();
  }

  Framebuffer& getBuffer() { return *current_; }

private:
  void swap()
  {
    // Just switch the pointers.
    Framebuffer* temp = current_;
    current_ = next_;
    next_ = temp;
  }

  Framebuffer  buffers_[2];
  Framebuffer* current_;
  Framebuffer* next_;
};
```

Now `Scene` has two buffers, stored in the `buffers_` array. We don't directly reference them from the array. Instead, there are two members, `next_` and `current_`, that point into the array. When we draw, we draw onto the next buffer, referenced by `next_`. When the video driver needs to get the pixels, it always accesses the *other* buffer through `current_`.

This way, the video driver never sees the buffer that we're working on. The only remaining piece of the puzzle is the call to `swap()` when the scene is done drawing the frame. That swaps the two buffers by simply switching the `next_` and `current_` references. The next time the video driver calls `getBuffer()`, it will get the new buffer we just finished drawing and put our recently drawn buffer on screen. No more tearing or unsightly glitches.

Not just for graphics

The core problem that double buffering solves is state being accessed while it's being modified. There are two common causes of this. We've covered the first one with our graphics example — the state is directly accessed from code on another thread or interrupt.

There is another equally common cause, though: when the code *doing the modification* is accessing the same state that it's modifying. This can manifest in a variety of places, especially physics and AI where you have entities interacting with each other. Double-buffering is often helpful here too.

Artificial unintelligence

Let's say we're building the behavioral system for, of all things, a game based on slapstick comedy. The game has a stage containing a bunch of actors that run around and get up to various hijinks and shenanigans. Here's our base actor:

```
class Actor
{
public:
  Actor() : slapped_(false) {}

  virtual ~Actor() {}
  virtual void update() = 0;

  void reset()      { slapped_ = false; }
  void slap()       { slapped_ = true; }
  bool wasSlapped() { return slapped_; }

private:
  bool slapped_;
};
```

Every frame, the game is responsible for calling `update()` on the actor so that it has a chance to do some processing. Critically, from the user's perspective, *all actors should appear to update simultaneously*.

> This is an example of the Update Method → pattern.

Actors can also interact with each other, if by "interacting", we mean "they can slap each other around". When updating, the actor can call `slap()` on another actor to slap it and call `wasSlapped()` to determine if it has been slapped.

The actors need a stage where they can interact, so let's build that:

```
class Stage
{
public:
  void add(Actor* actor, int index)
  {
    actors_[index] = actor;
  }

  void update()
  {
    for (int i = 0; i < NUM_ACTORS; i++)
    {
      actors_[i]->update();
      actors_[i]->reset();
    }
  }

private:
  static const int NUM_ACTORS = 3;

  Actor* actors_[NUM_ACTORS];
};
```

`Stage` lets us add actors, and provides a single `update()` call that updates each actor. To the user, actors appear to move simultaneously, but internally, they are updated one at a time.

The only other point to note is that each actor's "slapped" state is cleared immediately after updating. This is so that an actor only responds to a given slap once.

To get things going, let's define a concrete actor subclass. Our comedian here is pretty simple. He faces a single actor. Whenever he gets slapped—by anyone—he responds by slapping the actor he faces.

```
class Comedian : public Actor
{
public:
  void face(Actor* actor) { facing_ = actor; }

  virtual void update()
  {
    if (wasSlapped()) facing_->slap();
  }

private:
  Actor* facing_;
};
```

Now, let's throw some comedians on a stage and see what happens. We'll set up three comedians, each facing the next. The last one will face the first, in a big circle:

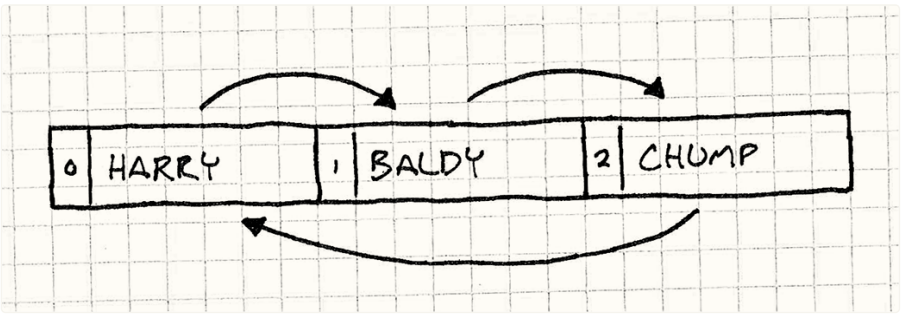
```
Stage stage;

Comedian* harry = new Comedian();
Comedian* baldy = new Comedian();
Comedian* chump = new Comedian();

harry->face(baldy);
baldy->face(chump);
chump->face(harry);

stage.add(harry, 0);
stage.add(baldy, 1);
stage.add(chump, 2);
```

The resulting stage is set up as shown in the following image. The arrows show who the actors are facing, and the numbers show their index in the stage's array.



We'll slap Harry to get things going and see what happens when we start processing:

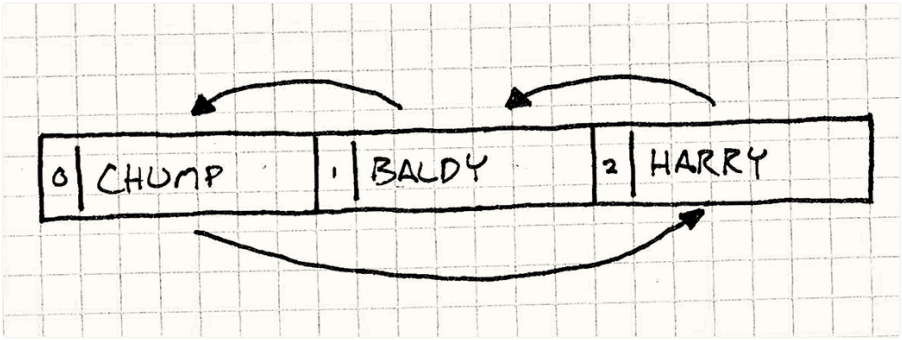
```
harry->slap();

stage.update();
```

Remember that the `update()` function in `Stage` updates each actor in turn, so if we step through the code, we'll find that the following occurs:

```
Stage updates actor 0 (Harry)
  Harry was slapped, so he slaps Baldy
Stage updates actor 1 (Baldy)
  Baldy was slapped, so he slaps Chump
Stage updates actor 2 (Chump)
  Chump was slapped, so he slaps Harry
Stage update ends
```

In a single frame, our initial slap on Harry has propagated through all of the comedians. Now, to mix things up a bit, let's say we reorder the comedians within the stage's array but leave them facing each other the same way.



We'll leave the rest of the stage setup alone, but we'll replace the chunk of code where we add the actors to the stage with this:

```
stage.add(harry, 2);
stage.add(baldy, 1);
stage.add(chump, 0);
```

Let's see what happens when we run our experiment again:

```
Stage updates actor 0 (Chump)
  Chump was not slapped, so he does nothing
Stage updates actor 1 (Baldy)
  Baldy was not slapped, so he does nothing
Stage updates actor 2 (Harry)
  Harry was slapped, so he slaps Baldy
Stage update ends
```

Uh, oh. Totally different. The problem is straightforward. When we update the actors, we modify their "slapped" states, the exact same state we also *read* during the update. Because of this, changes to that state early in the update affect later parts of that *same* update step.

> If you continue to update the stage, you'll see the slaps gradually cascade through the actors, one per frame. In the first frame, Harry slaps Baldy. In the next frame, Baldy slaps Chump, and so on.

The ultimate result is that an actor may respond to being slapped in either the *same* frame as the slap or in the *next* frame based entirely on how the two actors happen to be ordered on the stage. This violates our requirement that actors need to appear to run in parallel—the order that they update within a single frame shouldn't matter.

Buffered slaps

Fortunately, our Double Buffer pattern can help. This time, instead of having two copies of a monolithic "buffer" object, we'll be buffering at a much finer granularity: each actor's "slapped" state:

```
class Actor
{
public:
  Actor() : currentSlapped_(false) {}

  virtual ~Actor() {}
  virtual void update() = 0;

  void swap()
  {
    // Swap the buffer.
    currentSlapped_ = nextSlapped_;

    // Clear the new "next" buffer.
    nextSlapped_ = false;
  }

  void slap()       { nextSlapped_ = true; }
  bool wasSlapped() { return currentSlapped_; }

private:
  bool currentSlapped_;
  bool nextSlapped_;
};
```

Instead of a single `slapped_` state, each actor now has two. Just like the previous graphics example, the current state is used for reading, and the next state is used for writing.

The `reset()` function has been replaced with `swap()`. Now, right before clearing the swap state, it copies the next state into the current one, making it the new current state. This also requires a small change in `Stage`:

```
void Stage::update()
{
  for (int i = 0; i < NUM_ACTORS; i++)
  {
    actors_[i]->update();
  }

  for (int i = 0; i < NUM_ACTORS; i++)
  {
    actors_[i]->swap();
  }
}
```

The `update()` function now updates all of the actors and *then* swaps all of their states. The end result of this is that an actor will only see a slap in the frame *after* it was actually slapped. This way, the actors will behave the same no matter their order in the stage's array. As far as the user or any outside code can tell, all of the actors update simultaneously within a frame.

Design Decisions

Double Buffer is pretty straightforward, and the examples we've seen so far cover most of the variations you're likely to encounter. There are two main decisions that come up when implementing this pattern.

How are the buffers swapped?

The swap operation is the most critical step of the process since we must lock out all reading and modification of both buffers while it's occurring. To get the best

performance, we want this to happen as quickly as possible.

- **Swap pointers or references to the buffer:**

  This is how our graphics example works, and it's the most common solution for double-buffering graphics.

  - *It's fast.* Regardless of how big the buffer is, the swap is simply a couple of pointer assignments. It's hard to beat that for speed and simplicity.

  - *Outside code cannot store persistent pointers to the buffer.* This is the main limitation. Since we don't actually move the *data*, what we're essentially doing is periodically telling the rest of the codebase to look somewhere else for the buffer, like in our original stage analogy. This means that the rest of the codebase can't store pointers directly to data within the buffer — they may be pointing at the wrong one a moment later.

    This can be particularly troublesome on a system where the video driver expects the framebuffer to always be at a fixed location in memory. In that case, we won't be able to use this option.

  - *Existing data on the buffer will be from two frames ago, not the last frame.* Successive frames are drawn on alternating buffers with no data copied between them, like so:

    ```
    Frame 1 drawn on buffer A
    Frame 2 drawn on buffer B
    Frame 3 drawn on buffer A
    ...
    ```

    You'll note that when we go to draw the third frame, the data already on the buffer is from frame *one*, not the more recent second frame. In most cases, this isn't an issue — we usually clear the whole buffer right before drawing. But if we intend to reuse some of the existing data on the buffer, it's important to take into account that that data will be a frame older than we might expect.

    > One classic use of old framebuffer data is simulating motion blur. The current frame is blended with a bit of the previously rendered frame to make a resulting image that looks more like what a real camera captures.

- **Copy the data between the buffers:**

  If we can't repoint users to the other buffer, the only other option is to actually copy the data from the next frame to the current frame. This is how our slapstick comedians work. In that case, we chose this method because the state — a single Boolean flag — doesn't take any longer to copy than a pointer to the buffer would.

  - *Data on the next buffer is only a single frame old.* This is the nice thing about copying the data as opposed to ping-ponging back and forth between the two buffers. If we need access to previous buffer data, this will give us more up-to-date data to work with.

  - *Swapping can take more time.* This, of course, is the big negative point. Our swap operation now means copying the entire buffer in memory. If the buffer is large, like an entire framebuffer, it can take a significant chunk of time to do this. Since nothing can read or write to *either* buffer while this is happening, that's a big limitation.

What is the granularity of the buffer?

The other question is how the buffer itself is organized — is it a single monolithic chunk of data or distributed among a collection of objects? Our graphics example uses the former, and the actors use the latter.

Most of the time, the nature of what you're buffering will lead to the answer, but there's some flexibility. For example, our actors all could have stored their messages in a single message block that they all reference into by their index.

- **If the buffer is monolithic:**
  - *Swapping is simpler.* Since there is only one pair of buffers, a single swap does it. If you can swap by changing pointers, then you can swap the entire buffer, regardless of size, with just a couple of assignments.
- **If many objects have a piece of data:**
  - *Swapping is slower.* In order to swap, we need to iterate through the entire collection of objects and tell each one to swap.

    In our comedian example, that was OK since we needed to clear the next slap state anyway — every piece of buffered state needed to be touched each frame. If we don't need to otherwise touch the old buffer, there's a simple optimization we can do to get the same performance of a monolithic buffer while distributing the buffer across multiple objects.

    The idea is to get the "current" and "next" pointer concept and apply it to each of our objects by turning them into object-relative *offsets*. Like so:

```
class Actor
{
public:
  static void init() { current_ = 0; }
  static void swap() { current_ = next(); }

  void slap()        { slapped_[next()] = true; }
  bool wasSlapped()  { return slapped_[current_]; }

private:
  static int current_;
  static int next()  { return 1 - current_; }

  bool slapped_[2];
};
```

    Actors access their current slap state by using `current_` to index into the state array. The next state is always the other index in the array, so we can calculate that with `next()`. Swapping the state simply alternates the `current_` index. The clever bit is that `swap()` is now a *static* function — it only needs to be called once, and *every* actor's state will be swapped.

See Also

- You can find the Double Buffer pattern in use in almost every graphics API out there. For example, OpenGL has `swapBuffers()`, Direct3D has "swap chains", and Microsoft's XNA framework swaps the framebuffers within its `endDraw()` method.

© 2009-2014 Robert Nystrom